TRIBHUVAN UNIVERSITY

Faculty of Humanities and Social Sciences

Pashupati Multiple Campus

A Project Proposal on

ANKA: STUDENT RESULT ANALYSIS AND RANKING SYSTEM

Submitted to

Department of Computer Application

Pashupati Multiple Campus

In partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Project III (CACS452), Eighth Semester

Submitted by

Niranjan Katwal

TU Symbol No.: 27802063

TU Registration No.: 6-2-278-18-2022

Project Supervisor

Name and designation of the supervisor

Month and year of submission

CHAPTER 1: INTRODUCTION

1.1 Background

Every term, schools, colleges and universities in Nepal collect marks from many teachers, convert them into grades, work out each student's standing and announce results. In many institutions this work is still done with paper registers and separate spreadsheets. Marks are copied several times, grade points are calculated by hand, and the printed result is put on a notice board. Each copy step can introduce an error, and an error is usually found only after students complain.

Tribhuvan University (TU) follows a semester system for the Bachelor of Computer Applications (BCA). Every semester has subjects with credit hours, and each subject is assessed by internal marks and an external examination. A student who fails a subject sits a *back paper* in a later examination, and when it is passed the failure is cleared. The Nepal Examination Board (NEB) for grades 11 and 12 and the school boards use different grade-point tables and pass rules. An institution may run several programs under different rules at the same time.

This project proposes **Anka**, a web based Student Result Analysis and Ranking System. It stores syllabus, marks and attendance in one database, calculates semester and cumulative GPA with a configurable grading engine, handles back papers without losing history, ranks students, analyzes performance, warns about students at risk, sends result emails, verifies printed marksheets with a QR code, and supports daily class work through a timetable, class diary and assignments. The system carries no institution name, so any institution can use it.

1.2 Problem Statement

The way results and classes are managed in many institutions has the following problems:

- Results are slow and error-prone because marks are copied by hand between registers and spreadsheets.
- Back papers are difficult to track: it is unclear who owes which subject and how a cleared subject changes the semester GPA and the cumulative GPA.
- Different programs need different grading rules, but most tools support only one.
- Printed marksheets cannot be checked for authenticity.

- Weak students are noticed only after the final examination, when it is too late to help them.
- Students and guardians learn results from a notice board, and there is no record of emails or notices sent.
- Timetables, class diaries and assignment lists are scattered in notebooks and messaging groups.
- Local software is often tied to one institution through a hard-coded name or rule.

1.3 Objectives

1. To implement a configurable grading engine (TU semester GPA, NEB grade points, percentage divisions) with credit-weighted SGPA and CGPA.
2. To model back papers as separate sittings so that a cleared failure updates results while history is preserved.
3. To rank students with merge sort and to analyze performance with heap selection, statistics, regression, correlation and k-means clustering implemented from scratch.
4. To identify students at risk with an explainable score built from attendance, failures, GPA trend and internal marks.
5. To provide secure role based access for administrators, teachers, students and the public, and verification of marksheets by QR code.
6. To send emails through a queue with retries and a complete log.
7. To support daily class automation with a timetable, class diary and assignment tracking.
8. To keep the system neutral so that any institution can adopt it by adding its own details.

CHAPTER 2: METHODOLOGY

2.1 Requirement Identification

2.1.1 Study of Existing System

In the existing process, teachers record internal marks in a register, the board examination marks arrive on paper, the examination office copies everything into a spreadsheet and calculates grades, a printed result is posted on the notice board, and failed students are noted down for the next back paper examination. The study of this process, the TU rules and the CACS452 project syllabus produced the requirements below.

2.1.2 Functional Requirements

Table 2.1: Main functional requirements

No.	Requirement	Actor
1	Register, sign in, change and reset password; approve teacher accounts	All, administrator
2	Manage programs (grading scheme, pass mark, term names), subjects, students and teachers	Administrator
3	Enter marks by typing or CSV import; enter back paper marks	Teacher
4	Take attendance and compute the 80% eligibility	Teacher
5	Calculate, rank, publish and unpublish results; promote students	Administrator
6	View marksheet with QR code, transcript, attendance and insights	Student
7	Search results and verify marksheets without signing in; personal GPA checker	Public
8	Send and log result, reset, approval, notice and risk emails	System
9	Maintain timetable, class diary and assignments	Teacher, student
10	Optional institution branding; audit log	Administrator

2.1.3 Non-functional Requirements

- **Security:** hashed passwords, token based sessions, role checks on every route, rate limiting of public lookups.
- **Accuracy:** results must equal hand calculation; marks are validated and calculations run in transactions.
- **Usability:** clear navigation, responsive layout, light and dark themes.
- **Performance:** common pages within about one second; ranking in O(n log n).
- **Portability and maintainability:** runs on any operating system with Node.js and MySQL; grading rules are data.

2.1.4 Feasibility Study

Technical feasibility. All tools are open source and run on an ordinary laptop: Node.js, Express, React, Vite and MySQL. The developer has experience with JavaScript and databases. **Operational feasibility.** The system keeps the same roles and sequence of work as the manual process; teachers use a grid that behaves like a spreadsheet and students use the symbol number they already know. **Economic feasibility.** There are no licence costs; a small institution can host the system on an existing computer or a small virtual server. **Schedule feasibility.** The work is planned in seventeen weeks (Chapter 3).

2.2 High Level Design

2.2.1 System Architecture

Anka uses three tiers: a React single page client, a stateless Node.js and Express REST API, and a MySQL database. A mail worker reads queued messages from the database and sends them through Gmail SMTP.

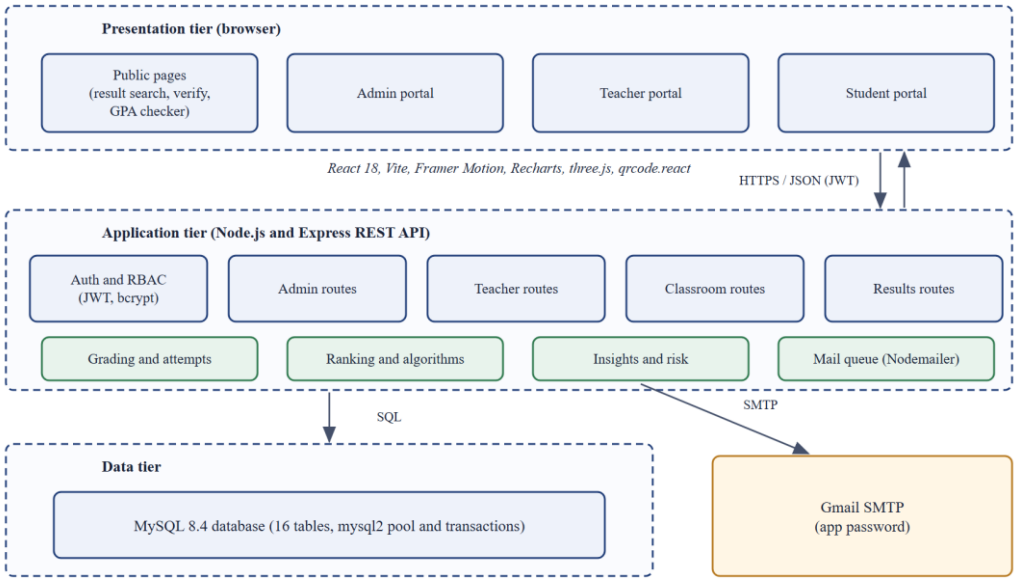


Figure 2.1: Proposed system architecture

2.2.2 Use Case Diagram

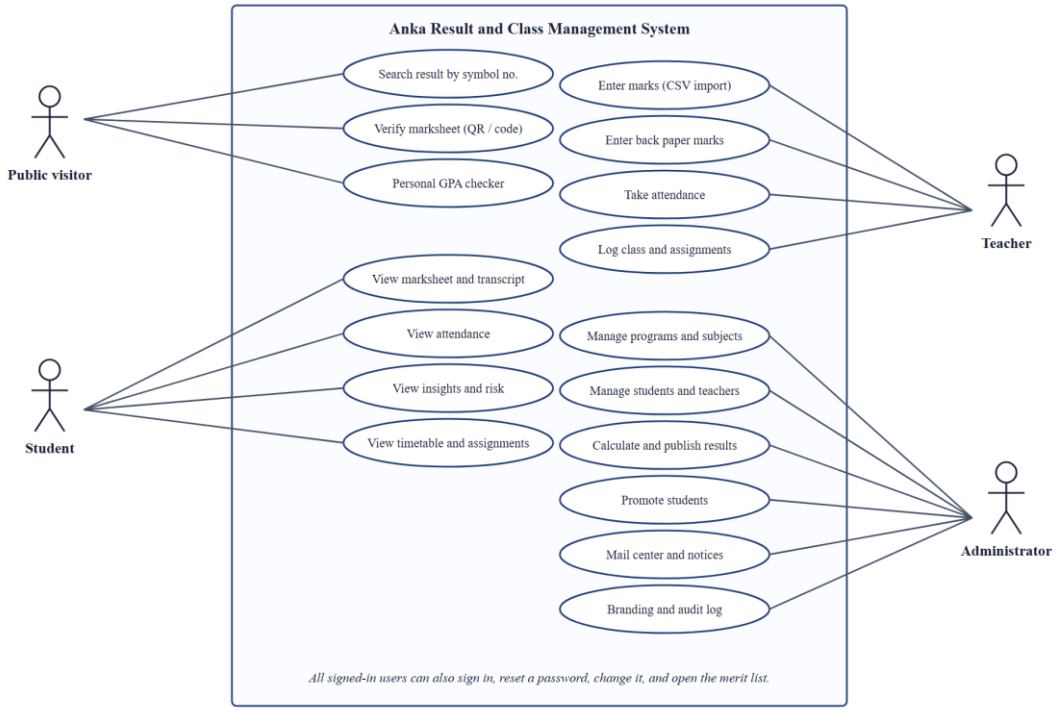


Figure 2.2: Use case diagram

Table 2.2: Use case: Enter back paper marks

Item	Description
Actor	Teacher
Precondition	The subject is assigned to the teacher and at least one student has not cleared it
Main flow	1. Open marks entry and choose back papers. 2. The system lists students who still owe the subject with every earlier sitting. 3. The teacher enters marks for the chosen year and saves. 4. The system stores a new attempt and recalculates the original semester result.
Postcondition	A passing sitting clears the backlog; GPA and CGPA are updated

2.2.3 Context Diagram

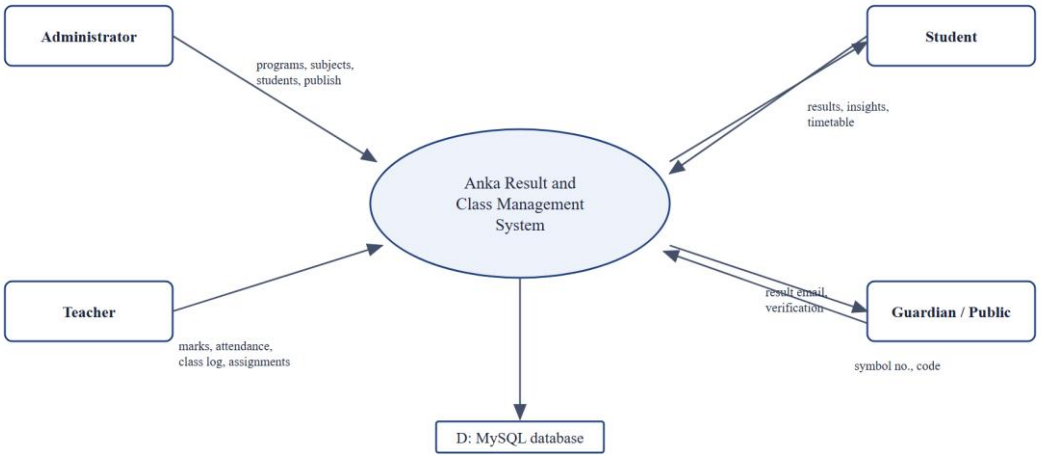


Figure 2.3: Context diagram (DFD level 0)

2.2.4 Data Flow Diagram (Level 1)

Level 1 breaks the system into six processes: academic setup, recording marks and back papers, calculating results, publishing and notifying, analysis and ranking, and daily class automation.

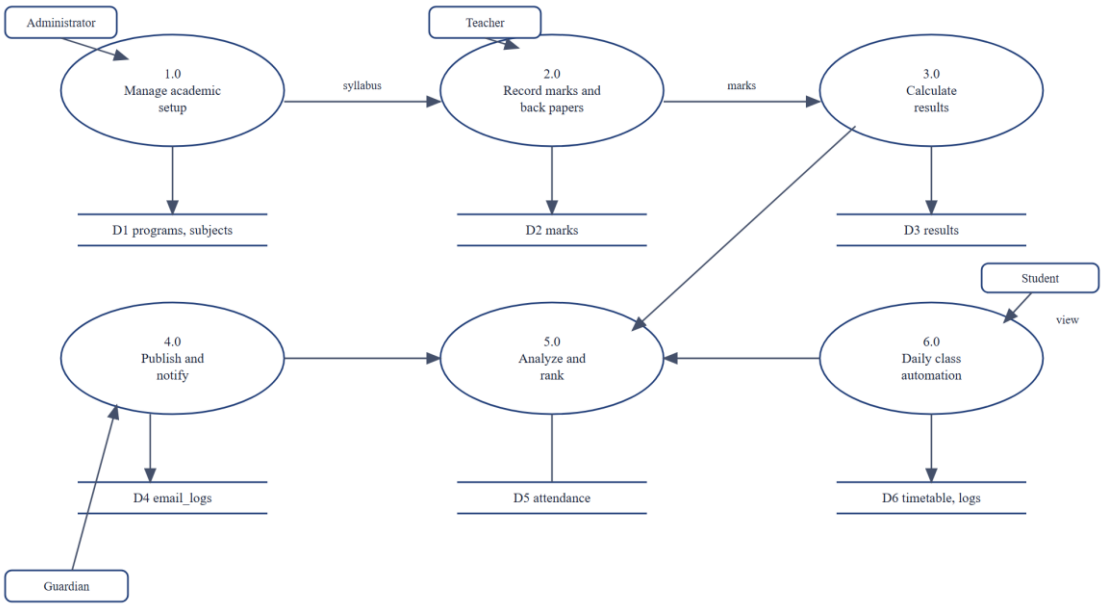


Figure 2.4: DFD level 1

2.2.5 Class Diagram

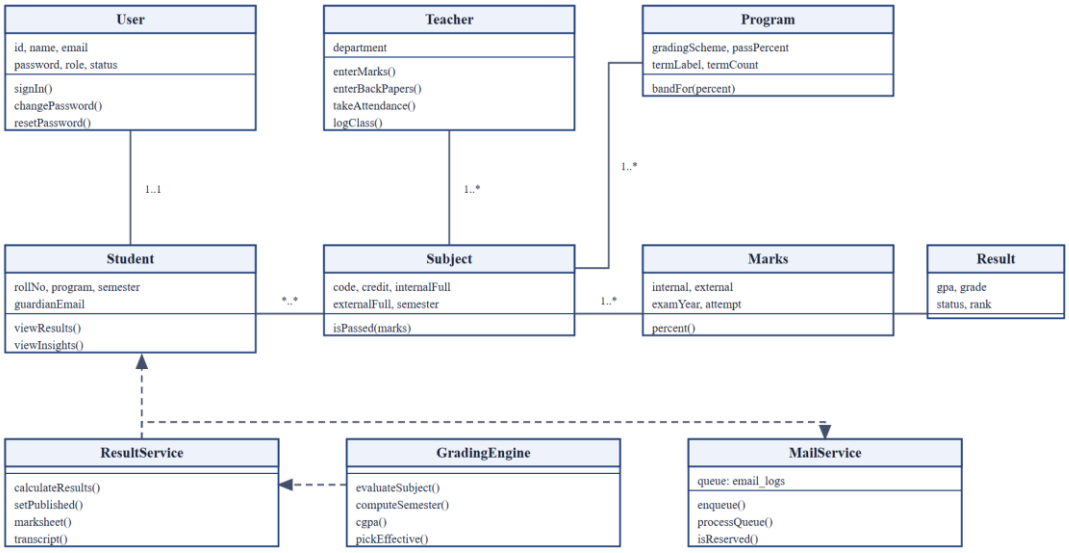


Figure 2.5: Class diagram of the main classes

2.2.6 Activity of Result Calculation

The administrator chooses a program, term and exam year. The system loads the cohort and every later sitting, picks the effective attempt of each subject, computes grades and GPA, decides pass or fail, ranks the cohort and saves the results.

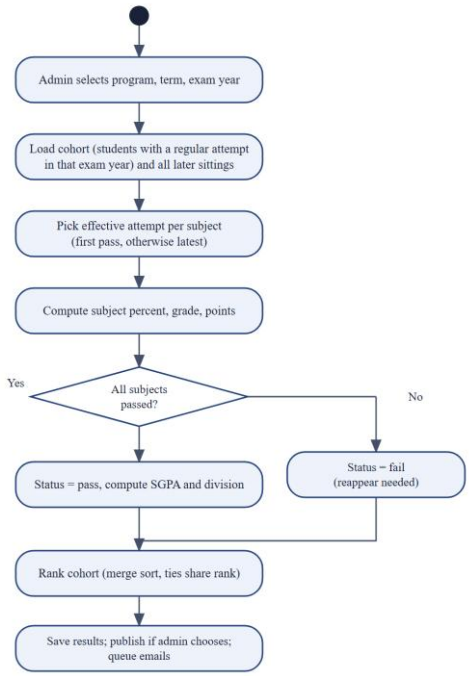


Figure 2.6: Activity diagram: calculate and publish results

2.2.7 Database Design

The database has sixteen tables: programs, users, teachers, students, subjects, marks, results, attendance, timetable, class_logs, assignments, assignment_status, notices, email_logs, audit_logs and settings. The `marks` table stores one row per sitting with an `attempt` number, which is how back papers are modeled. The figure below shows the entity relationship diagram.

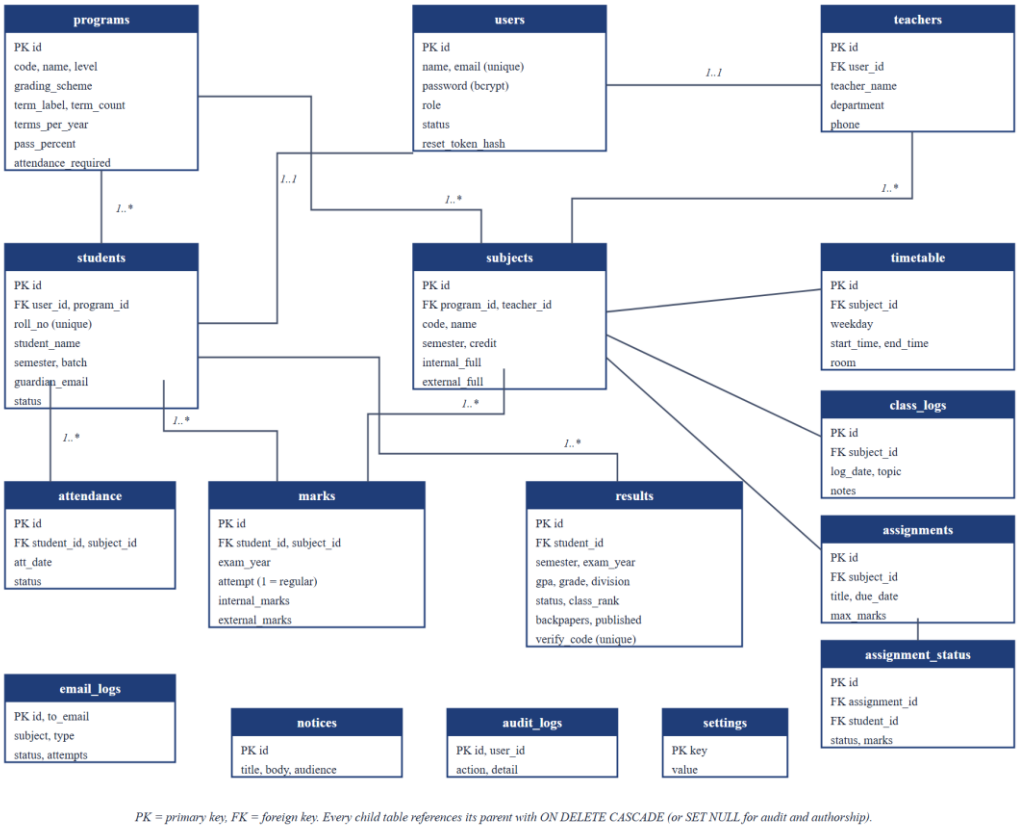


Figure 2.7: Entity relationship diagram

2.2.8 Algorithms

Table 2.3: Algorithms to be implemented from scratch

Algorithm	Used for	Complexity
Binary search	Grade band lookup, percentile of a student	O(log n)
Merge sort with competition ranking	Merit list and class rank with ties	O(n log n)
Binary min-heap (top-k)	Top performers of a subject	O(n log k)
Descriptive statistics, z-score	Mean, median, deviation, pass rate	O(n log n)
Least squares regression	GPA trend and next-term forecast	O(n)
Pearson correlation	Attendance against GPA	O(n)
K-means clustering	Grouping students for support	O(n k i)

Algorithm	Used for	Complexity
Weighted risk score	Explainable early warning	O(1)
SHA-256 token and bcrypt hash	Password reset links and passwords	Library primitives

2.2.9 Back Paper Handling

A failed subject becomes a backlog. When the student sits the back paper, a new row with a higher attempt number is stored. The result calculation chooses the effective attempt (the first passing sitting, otherwise the latest) and recalculates the original semester GPA and the CGPA. Every earlier sitting stays in the history, and published results stay published.

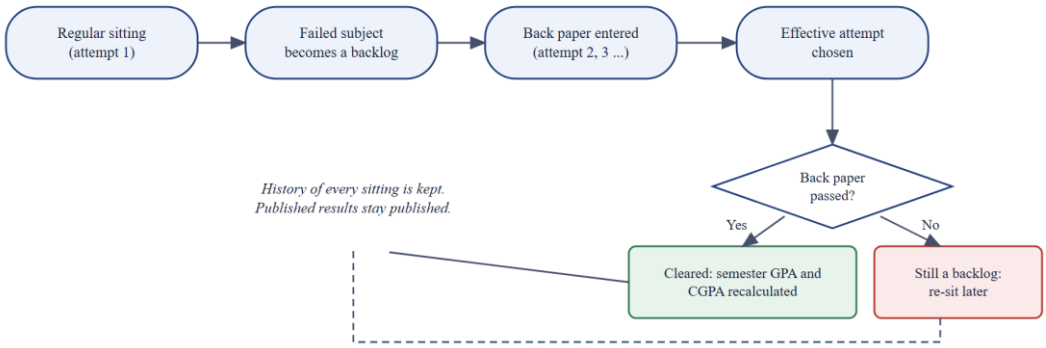


Figure 2.8: Life cycle of a failed subject and its back paper

2.2.10 Interface Design

The interface will use one design system with a neutral navy palette, shared components, light and dark themes, a command palette and a collapsible sidebar. The pages for each role are planned as follows.

Table 2.4: Planned pages by role

Role	Pages
Public	Landing, result search, marksheet verification, GPA checker, algorithms explorer, sign in, register, reset password
Student	Dashboard, classes, results and transcript, attendance, insights, merit list
Teacher	My subjects, classes, marks entry and back papers, attendance, class insights
Administrator	Dashboard, insights, programs, subjects, students, teachers, classes, results, mail center, notices, branding, audit log

2.3 Implementation and Testing Plan

2.3.1 Tools

Table 2.5: Proposed tools

Area	Tools
Frontend	React 18, Vite, Framer Motion, Recharts, three.js, qrcode.react
Backend	Node.js, Express, mysql2, bcryptjs, jsonwebtoken, nodemailer
Database	MySQL 8.4 (portable copy inside the project)
Version control	Git and GitHub
Testing	node:test, API flow scripts, browser tests with puppeteer-core

2.3.2 Process Model

An incremental and iterative model will be followed: each cycle adds one slice of functionality, tests it and fixes the defects found, then the next slice is started. This suits a single developer who must keep the whole system consistent.

2.3.3 Testing

Unit tests will cover grading, ranking and the algorithms. API flow tests will cover every endpoint and a role by endpoint permission matrix. Browser tests will drive the real pages at desktop, tablet and phone widths. The email pipeline will be tested against a local SMTP server so that no real message is sent during tests.

2.3.4 Test Plan

Table 2.6: Test plan

Level	What is checked	Tool
Unit	Grade lookup, GPA, effective attempt, ranking, heap, regression, k-means, risk score, email queue	node:test
API flows	Every endpoint, validation, permission matrix, back papers, publish and promote	Node scripts
Smoke	Key numbers on seeded data	Node scripts
Browser	Real pages, forms, dialogs, shortcuts, responsive widths, dark theme	puppeteer-core

2.4 Risks and Assumptions

Table 2.7: Project risks and mitigation

Risk	Mitigation
Official TU grade table or back paper rule differs from the assumed scheme	Keep scales and rules as editable data; confirm with the department

Risk	Mitigation
Gmail limits or blocks automatic mail	Queue with retries and a log; messages can be previewed without sending
Single developer and limited time	Incremental delivery; most important features first; automated tests to protect existing work
Data privacy of student results	Role checks, hashed passwords, public lookup limited to symbol number and verification code

CHAPTER 3: GANTT CHART

The work is planned over seventeen weeks as shown in Figure 3.1. Analysis and design start early and overlap with implementation, and testing runs alongside the later implementation work.

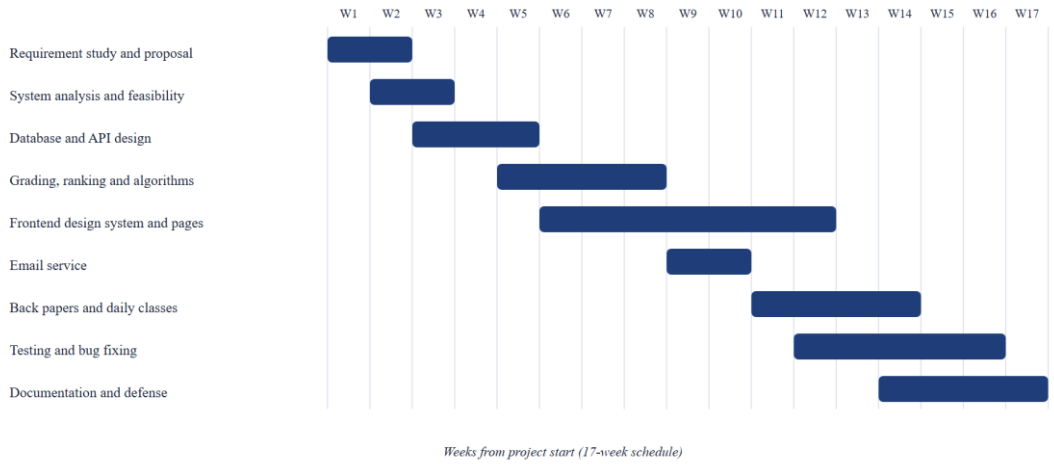


Figure 3.1: Project schedule

Table 3.1: Schedule by activity

Activity	Weeks	Deliverable
Requirement study and proposal	1 to 2	Proposal
System analysis and feasibility	2 to 3	DFD, ER diagram, use cases
Database and API design	3 to 5	Schema, route list
Grading, ranking and algorithms	5 to 8	Grading engine, tests
Frontend design system and pages	6 to 12	Role based portals
Email service	9 to 10	Mail queue and templates
Back papers and daily classes	11 to 14	Back paper and classroom modules
Testing and bug fixing	12 to 16	Test suites, fixes
Documentation and defense	14 to 17	Report, proposal, presentation

CHAPTER 4: EXPECTED OUTCOME

At the end of the project the following outcomes are expected:

- A working web application with four kinds of users and about eighty REST endpoints.
- A grading engine that produces correct SGPA and CGPA for TU, NEB and division based programs, and handles back papers without losing history.
- Result publication with ranking, marksheets with QR code verification, and email notification with a log.
- Analytics and an explainable early-warning score for students at risk.
- Daily class tools: timetable with clash detection, class diary and assignments.
- A neutral, brandable product that any institution in Nepal can adopt.
- Automated tests, a project report, this proposal and a presentation.

REFERENCES

[1] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601-618, Nov. 2010.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[3] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, vol. 1, 1967, pp. 281-297.

[4] N. Provos and D. Mazieres, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81-91.

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[6] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[7] Tribhuvan University, FOHSS, "CACS452 Project III, BCA eighth semester course outline." [Online]. Available: https://www.itcollegenepal.com/wp-content/uploads/2022/02/CACS452-BCA-VIII-SEM-Project-III.pdf. [Accessed: 2 Nov. 2026].

[8] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev. [Accessed: 2 Nov. 2026].

[9] Oracle Corporation, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. [Accessed: 2 Nov. 2026].